Compte rendu TD2

Exercice2 :

1.L'organisation optimale des livraisons :

L'offre totale (8 + 9 = 17) est égale à la demande totale (4 + 5 + 8 = 17) : le problème est équilibré.

L'organisation optimale est la suivante : le dépôt D1 livre 3 containers au magasin M1 et 5 containers au magasin M2 ; le dépôt D2 livre 1 container au magasin M1 et 8 containers au magasin M3

Coût total minimal = 15 + 15 + 6 + 16 = 52

2.Modélisation (programmation linéaire en nombres entiers) :

Variables de décision x_{ij} = nombre de containers transportés du dépôt Di vers le magasin Mj (i = 1, 2 ; j = 1, 2, 3), avec x_{ij} entier et $x_{ij} \geq 0$.

Fonction objectif = min Z = 5x11 + 3x12 + 4x13 + 6x21 + 7x22 + 2x23

Contraintes=

- Offre : x11 + x12 + x13 = 8 et x21 + x22 + x23 = 9
- Demande : x11 + x21 = 4, x12 + x22 = 5 et x13 + x23 = 8

Ce problème peut être résolu de façon exacte par Branch and Bound. Ici, nous le résolvons par la méthode heuristique de la recherche taboue.

3. Principe de la recherche taboue appliquée au problème :

- **Solution initiale (coin nord-ouest) :**On remplit le tableau à partir de la case en haut à gauche en envoyant le maximum possible, puis on passe à la case suivante. On obtient D1 = [4, 4, 0] et D2 = [0, 1, 8], de coût 4×5 + 4×3 + 1×7 + 8×2 = 55.
- **Voisinage :** Un voisin est obtenu par un mouvement rectangulaire 2×2 : D1 envoie un container de moins vers un magasin et un de plus vers un autre, et D2 fait l'inverse. Les offres et les demandes restent respectées, donc tout voisin est une solution réalisable.
- **Liste tabou :** Après chaque mouvement, le mouvement inverse est interdit pendant 4 itérations (taille de la liste = 4), afin d'éviter de revenir en arrière et de tourner en cycle.

- **Critère d'aspiration :**Un mouvement tabou est accepté s'il donne un coût meilleur que le meilleur coût trouvé jusque-là.
- **Choix du voisin :** À chaque itération, on choisit le meilleur voisin autorisé, même s'il est moins bon que la solution courante. C'est ce qui permet de sortir d'un optimum local.
- **Arrêt :** Après 15 itérations ,on conserve la meilleure solution rencontrée**.**

4. Déroulement des itérations

Dans la colonne « Mouvement », « M1→M2 » signifie que D1 envoie un container de moins à M1 et un de plus à M2 (D2 fait l'inverse).

Itération	Mouvement	D1 (M1, M2, M3)	D2 (M1, M2, M3)	Coût	Meilleur coût
0	initiale	4, 4, 0	0, 1, 8	55	55
1	M1→M2	3, 5, 0	1, 0, 8	52	52
2	M1→M3	2, 5, 1	2, 0, 7	55	52
3	M1→M3	1, 5, 2	3, 0, 6	58	52
4	M1→M3	0, 5, 3	4, 0, 5	61	52
5	M2→M3	0, 4, 4	4, 1, 4	67	52
6	M2→M1	1, 3, 4	3, 2, 4	70	52
7	M2→M1	2, 2, 4	2, 3, 4	73	52
8	M2→M1	3, 1, 4	1, 4, 4	76	52
9	M3→M1	4, 1, 3	0, 4, 5	73	52
10	M3→M2	4, 2, 2	0, 3, 6	67	52
11	M3→M2	4, 3, 1	0, 2, 7	61	52
12	M3→M2	4, 4, 0	0, 1, 8	55	52
13	M1→M2	3, 5, 0	1, 0, 8	52	52
14	M1→M3	2, 5, 1	2, 0, 7	55	52
15	M1→M3	1, 5, 2	3, 0, 6	58	52

Remarque : À l'itération 1, on trouve déjà le meilleur coût (52). Aux itérations suivantes, l'algorithme accepte des solutions plus coûteuses (55, 58, 61...) pour explorer d'autres régions, puis revient à 52 à l'itération 13.

5. Résultat : organisation optimale des livraisons

Livraison	Containers	Coût unitaire	Sous-total
D1 → M1	3	5	15
D1 → M2	5	3	15
D2 → M1	1	6	6
D2 → M3	8	2	16

Le dépôt D1 livre 3 containers au magasin M1 et 5 containers au magasin M2 ; le dépôt D2 livre 1 container au magasin M1 et 8 containers au magasin M3. Les liaisons D1→M3 et D2→M2 ne sont pas utilisées.

Coût total minimal = 15 + 15 + 6 + 16 = 52.

6.conculsion :

La recherche taboue a permis de trouver la solution optimale du problème de transport, de coût 52. La liste tabou permet de quitter les optima locaux, ce qu'une simple descente ne permet pas. Pour de grandes instances, Branch and Bound donne l'optimum exact mais devient très coûteux en temps de calcul, alors que la recherche taboue fournit rapidement une bonne solution.

Code python :

Code source sur GitHub :**https://github.com/nourelhoudahawem/TD2_compte-rendu/blob/main/exercice2.py**

```
PS C:\Users\ASUS TUF\Desktop\algorithmes évolutionnaires\TD2>    python exercice2.py
Init : [[4, 4, 0], [0, 1, 8]] coût = 55
Iter 1 : [[3, 5, 0], [1, 0, 8]] coût = 52 | meilleur = 52 | tabou = [(0, 1, -1)]
Iter 2 : [[2, 5, 1], [2, 0, 7]] coût = 55 | meilleur = 52 | tabou = [(0, 1, -1), (0, 2, -1)]
Iter 3 : [[1, 5, 2], [3, 0, 6]] coût = 58 | meilleur = 52 | tabou = [(0, 1, -1), (0, 2, -1), (0, 2, -1)]
Iter 4 : [[0, 5, 3], [4, 0, 5]] coût = 61 | meilleur = 52 | tabou = [(0, 1, -1), (0, 2, -1), (0, 2, -1), (0, 2, -1)]
Iter 5 : [[0, 4, 4], [4, 1, 4]] coût = 67 | meilleur = 52 | tabou = [(0, 2, -1), (0, 2, -1), (0, 2, -1), (1, 2, -1)]
Iter 6 : [[1, 3, 4], [3, 2, 4]] coût = 70 | meilleur = 52 | tabou = [(0, 2, -1), (0, 2, -1), (1, 2, -1), (0, 1, 1)]
Iter 7 : [[2, 2, 4], [2, 3, 4]] coût = 73 | meilleur = 52 | tabou = [(0, 2, -1), (1, 2, -1), (0, 1, 1), (0, 1, 1)]
Iter 8 : [[3, 1, 4], [1, 4, 4]] coût = 76 | meilleur = 52 | tabou = [(1, 2, -1), (0, 1, 1), (0, 1, 1), (0, 1, 1)]
Iter 9 : [[4, 1, 3], [0, 4, 5]] coût = 73 | meilleur = 52 | tabou = [(0, 1, 1), (0, 1, 1), (0, 1, 1), (0, 2, 1)]
Iter 10 : [[4, 2, 2], [0, 3, 6]] coût = 67 | meilleur = 52 | tabou = [(0, 1, 1), (0, 1, 1), (0, 2, 1), (1, 2, 1)]
Iter 11 : [[4, 3, 1], [0, 2, 7]] coût = 61 | meilleur = 52 | tabou = [(0, 1, 1), (0, 2, 1), (1, 2, 1), (1, 2, 1)]
Iter 12 : [[4, 4, 0], [0, 1, 8]] coût = 55 | meilleur = 52 | tabou = [(0, 2, 1), (1, 2, 1), (1, 2, 1), (1, 2, 1)]
Iter 13 : [[3, 5, 0], [1, 0, 8]] coût = 52 | meilleur = 52 | tabou = [(1, 2, 1), (1, 2, 1), (1, 2, 1), (0, 1, -1)]
Iter 14 : [[2, 5, 1], [2, 0, 7]] coût = 55 | meilleur = 52 | tabou = [(1, 2, 1), (1, 2, 1), (0, 1, -1), (0, 2, -1)]
Iter 15 : [[1, 5, 2], [3, 0, 6]] coût = 58 | meilleur = 52 | tabou = [(1, 2, 1), (0, 1, -1), (0, 2, -1), (0, 2, -1)]

===== Solution finale =====
D1 -> M1: 3  M2: 5  M3: 0
D2 -> M1: 1  M2: 0  M3: 8
Coût minimal = 52
```

Exécution du programme exercice2.py

Exercice3 :

Solution initiale imposée : 1-2-3-4-5-6-7-8-1, de longueur 14 + 13 + 11 + 11 + 15 + 9 + 13 + 17 = **103**.

Voisinage utilisé (2-opt) : on choisit deux positions i < j dans la tournée (la ville 1 reste fixée en tête) et on inverse le segment compris entre ces deux positions.

1 . recherche taboue :

Paramètres : liste tabou de taille 5 (on mémorise les inversions (i, j) déjà effectuées), 20 itérations. À chaque itération on choisit le meilleur voisin non tabou, même s'il est moins bon que la tournée courante ; un mouvement tabou n'est accepté que s'il bat le meilleur résultat connu (aspiration).

Itération	Inversion (i, j)	Tournée obtenue	Longueur	Meilleure longueur
0	initiale	1-2-3-4-5-6-7-8-1	103	103
1	(1, 7)	1-8-7-6-5-4-3-2-1	103	103
2	(6, 7)	1-8-7-6-5-4-2-3-1	107	103
3	(1, 6)	1-2-4-5-6-7-8-3-1	100	100
4	(6, 7)	1-2-4-5-6-7-3-8-1	98	98
5	(2, 3)	1-2-5-4-6-7-3-8-1	99	98
6	(3, 4)	1-2-5-6-4-7-3-8-1	102	98
7	(1, 7)	1-8-3-7-4-6-5-2-1	102	98
8	(4, 5)	1-8-3-7-6-4-5-2-1	99	98
9	(5, 6)	1-8-3-7-6-5-4-2-1	98	98
10	(1, 2)	1-3-8-7-6-5-4-2-1	100	98
11	(2, 7)	1-3-2-4-5-6-7-8-1	107	98
12	(3, 4)	1-3-2-5-4-6-7-8-1	108	98
13	(1, 7)	1-8-7-6-4-5-2-3-1	108	98
14	(1, 6)	1-2-5-4-6-7-8-3-1	101	98
15	(6, 7)	1-2-5-4-6-7-3-8-1	99	98
16	(2, 3)	1-2-4-5-6-7-3-8-1	98	98
17	(3, 5)	1-2-4-7-6-5-3-8-1	107	98
18	(2, 5)	1-2-5-6-7-4-3-8-1	103	98
19	(4, 5)	1-2-5-6-4-7-3-8-1	102	98
20	(3, 4)	1-2-5-4-6-7-3-8-1	99	98

Le premier mouvement (1, 7) inverse toute la tournée : il donne le même cycle parcouru en sens inverse, donc la même longueur (103).

Résultat (tabou) : la meilleure tournée est **1-2-4-5-6-7-3-8-1**, de longueur **98**, trouvée dès l'itération 4.

Détail : 14 (1-2) + 14 (2-4) + 11 (4-5) + 15 (5-6) + 9 (6-7) + 9 (7-3) + 9 (3-8) + 17 (8-1) = 98.

Code python :

Code source sur GitHub :https://github.com/nourelhoudahawem/TD2_compte-rendu/blob/main/exercice3.py (tabou+recuit simulé)

```
PS C:\Users\ASUS TUF\Desktop\algorithmes évolutionnaires\TD2>    python exercice3.py
=== TABOU ===
Init 1-2-3-4-5-6-7-8-1  L = 103
Iter  1 : inversion (1, 7) -> 1-8-7-6-5-4-3-2-1  L = 103 | meilleur = 103
Iter  2 : inversion (6, 7) -> 1-8-7-6-5-4-2-3-1  L = 107 | meilleur = 103
Iter  3 : inversion (1, 6) -> 1-2-4-5-6-7-8-3-1  L = 100 | meilleur = 100
Iter  4 : inversion (6, 7) -> 1-2-4-5-6-7-3-8-1  L = 98 | meilleur = 98
Iter  5 : inversion (2, 3) -> 1-2-5-4-6-7-3-8-1  L = 99 | meilleur = 98
Iter  6 : inversion (3, 4) -> 1-2-5-6-4-7-3-8-1  L = 102 | meilleur = 98
Iter  7 : inversion (1, 7) -> 1-8-3-7-4-6-5-2-1  L = 102 | meilleur = 98
Iter  8 : inversion (4, 5) -> 1-8-3-7-6-4-5-2-1  L = 99 | meilleur = 98
Iter  9 : inversion (5, 6) -> 1-8-3-7-6-5-4-2-1  L = 98 | meilleur = 98
Iter 10 : inversion (1, 2) -> 1-3-8-7-6-5-4-2-1  L = 100 | meilleur = 98
Iter 11 : inversion (2, 7) -> 1-3-2-4-5-6-7-8-1  L = 107 | meilleur = 98
Iter 12 : inversion (3, 4) -> 1-3-2-5-4-6-7-8-1  L = 108 | meilleur = 98
Iter 13 : inversion (1, 7) -> 1-8-7-6-4-5-2-3-1  L = 108 | meilleur = 98
Iter 14 : inversion (1, 6) -> 1-2-5-4-6-7-8-3-1  L = 101 | meilleur = 98
Iter 15 : inversion (6, 7) -> 1-2-5-4-6-7-3-8-1  L = 99 | meilleur = 98
Iter 16 : inversion (2, 3) -> 1-2-4-5-6-7-3-8-1  L = 98 | meilleur = 98
Iter 17 : inversion (3, 5) -> 1-2-4-7-6-5-3-8-1  L = 107 | meilleur = 98
Iter 18 : inversion (2, 5) -> 1-2-5-6-7-4-3-8-1  L = 103 | meilleur = 98
Iter 19 : inversion (4, 5) -> 1-2-5-6-4-7-3-8-1  L = 102 | meilleur = 98
Iter 20 : inversion (3, 4) -> 1-2-5-4-6-7-3-8-1  L = 99 | meilleur = 98
Meilleure tournée : 1-2-4-5-6-7-3-8-1  L = 98
```

Exécution du programme exercice3.py(tabou)

2 . recuit simulé :

Principe : On part de la tournée initiale. À chaque essai, on tire au hasard un voisin (inversion de deux positions). Soit Δ = longueur(voisin) − longueur(courante) :

- si Δ ≤ 0, on accepte le voisin ;
- sinon, on l'accepte avec la probabilité exp(−Δ / T) (critère de Metropolis).

Au début la température T est élevée et le recuit accepte souvent des dégradations, ce qui évite de rester bloqué dans un optimum local ; en refroidissant, il devient de plus en plus sélectif.

Paramètres : 5 itérations (paliers de température) avec 15 essais par palier, température initiale T0 = 30 et variation empirique de la température T ← 0,6 × T (30 ; 18 ; 10,8 ; 6,48 ; 3,888). Graine aléatoire fixée à 3 pour reproduire les résultats.

Itération	Température T	Tournée courante (fin de palier)	Longueur	Meilleure longueur
0	–	1-2-3-4-5-6-7-8-1	103	103
1	30	1-3-6-4-2-5-7-8-1	124	103
2	18	1-3-7-6-5-4-2-8-1	111	103
3	10,8	1-3-7-6-5-4-2-8-1	111	103
4	6,48	1-2-4-5-6-7-3-8-1	98	98
5	3,888	1-8-3-7-6-5-4-2-1	98	98

Résultat (recuit simulé) : la meilleure tournée est **1-2-4-5-6-7-3-8-1**, de longueur **98** (la tournée de l'itération 5 est la même, parcourue en sens inverse).

Le recuit simulé étant une méthode aléatoire, le résultat dépend du tirage : avec une autre graine ou d'autres paramètres, il peut rester sur une solution moins bonne. Les paliers à haute température (itérations 1 à 3) ont accepté des tournées plus longues (124, 111), puis le refroidissement a permis de converger vers 98.

```
=== RECUIT SIMULE ===
Init 1-2-3-4-5-6-7-8-1  L = 103  T0 = 30.0
Iter 1 : T =  30.000  courant = 1-3-6-4-2-5-7-8-1 (L=124)  meilleur = 103
Iter 2 : T =  18.000  courant = 1-3-7-6-5-4-2-8-1 (L=111)  meilleur = 103
Iter 3 : T =  10.800  courant = 1-3-7-6-5-4-2-8-1 (L=111)  meilleur = 103
Iter 4 : T =   6.480  courant = 1-2-4-5-6-7-3-8-1 (L=98)  meilleur = 98
Iter 5 : T =   3.888  courant = 1-8-3-7-6-5-4-2-1 (L=98)  meilleur = 98
Meilleure tournée : 1-2-4-5-6-7-3-8-1  L = 98
```

Exécution du programme exercice3.py(Recuit Simule)

4. Comparaison et conclusion

Méthode	Meilleure tournée	Longueur
Solution initiale	1-2-3-4-5-6-7-8-1	103
Recherche taboue	1-2-4-5-6-7-3-8-1	98
Recuit simulé	1-2-4-5-6-7-3-8-1	98

Les deux méthodes améliorent la solution initiale (103 → 98). Une énumération de toutes les tournées possibles confirme que **98 est l'optimum** de ce problème.

La recherche taboue est déterministe et explore systématiquement le voisinage en s'appuyant sur sa mémoire (liste tabou). Le recuit simulé est stochastique et

évite les optima locaux grâce à l'acceptation probabiliste de solutions moins bonnes à haute température. Sur ce petit problème, les deux atteignent l'optimum.